

kernels. This is followed by the information on which kernels are to be run in software and which in hardware, area-speed tradeoff best for each of the hardware kernels and optimal configuration (number of cores, cache sizes, memory banks, etc) for the multi-core processor. This is done by carefully examining through several simulation and synthesis runs.

The simulation tool provides runtime estimates for execution of each kernel in software, for several configurations of the multi-core processor (with varying cache sizes, memory banks, etc). The synthesis tool provides latency estimates for execution of each kernel in hardware, with varying hardware footprints (trading area for speed). The design space exploration tool uses these area and performance estimates, along with its full knowledge of the underlying platform's resources and available configurations, to heuristically search for the best answers to the questions listed above. User experience can be used to guide the tool, e.g., by restricting the search to a smaller set with the most 'interesting' multi-core configurations.

Multi-core processor

The CUDA host program as well as software kernels, i.e., the subset of kernels determined by the design space exploration tool, run in software on the multi-core processor (Fig. 9).

It uses Xilinx Microblaze soft cores with separate instruction caches and a shared data cache, all communicating through two AXI4-based buses. FASTCuda follows a similar mapping of the threads with a GPU. Each core executes thread-block, which can use the core's scratchpad memory as a private local memory. All the threads from any thread block can access

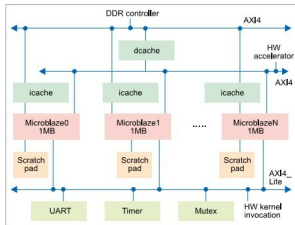


Fig. 9: Multi-core processor

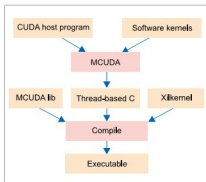


Fig. 10: CUDA kernel porting process using MCUDA

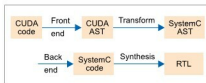


Fig. 11: CUDA kernels in hardware

the global shared memory, which can also be accessed by the hardware accelerator. The AXI4_Lite bus is used for communication between the multi-core processor and the accelerator block that is running hardware kernels. A simple handshake protocol is employed to pass the arguments and trigger a specific hardware kernel to start running, which will then respond back when it has finished running. Lastly, the timer and mutex blocks on the AXI4_Litebus are a requirement for symmetric multiprocessing support of the runtime on the processor.

Implementing CUDA kernels on the multi-processor

The OS-level software running on the multi-core processor here is a modified version of the Xilinx kernel 'Xilkernel'. Xilkernel supports POSIX threads, mutexes and semaphores, but was meant to run on a single core, thus having no support for an SMP environment like the one here. We consequently had to add SMP support to Xilkernel.

CUDA kernels are supposed to run on SIMT devices (GPUs), which are drastically different from multi-core processors. Thus, the next step is to port the CUDA kernels, using MCUDA, to run on top of the multi-core multi-threaded environment provided by modified Xilkernel.

MCUDA transforms the CUDA code into a thread-based C code that uses MCUDA library in order to create a pool of threads and coordinate thread operations as well as provide the basic CUDA runtime functionality for kernel invocation and data movements. Xilkernel provides the mutex support required by MCUDA library and the thread-based support required by multi-threaded software kernels. In CUDA the host program is usually run on a chip separate from the CUDA kernels; the first is run on a general-purpose CPU and the latter on a GPU. Thus the CUDA programming model assumes that the host and device maintain their own separate memory spaces, referred to as host memory and device memory, respectively.

The execution of a kernel involves:

1. Memory transfers of input vectors from the host memory to the device memory,
2. Kernel execution, which uses input vectors in order to generate output vectors and